>Maestro CLI
User Guide

M3UG-01

November 2025

Version 1

Introduction

`Maestro CLI`, also known as `M3-CLI`, is a robust command-line interface (CLI) tool designed to provide a powerful alternative to user interfaces (UI). It allows you to perform cloud infrastructure management operations, as well as manage related assets, by sending respective calls to Maestro cloud management platform. The guide is aimed at Maestro users who plan to manage their virtual infrastructure using the CLI interface. Here, you'll find all the resources you need to install, update, uninstall, and effectively use the Maestro CLI.

Table of Contents

1. Setup Guide
    - Install
    - Update & Healthcheck
    - Uninstall
2. Usage Guide
    - Configuration
    - Usage Examples
3. Support
    - How to Report an Issue
    - How to Communicate with the Support Team
4. Annexes
    - Annex 1: Common Use Cases
    - Annex 2: Developers Guide

We're here to help you every step of the way. If you encounter any issues or have any questions, don't hesitate to reach out to our support team.

M3-CLI Setup Guide

Tool Installation

Pre-requisites

Before you start with the installation of `M3-CLI`, ensure you have **Python 3.10+**, **pip** and **venv** installed on your system. If not, follow the guides provided below:

- Python3 Installation Guide for Windows
- Python3 Installation Guide for Linux
- Python3 Installation Guide for Mac

Installation Flow

> **Tip: It's recommended to use a virtual environment to protect against dependency conflicts. You can install the `m3cli` tool without this step, but if you install it in a virtual environment, you will need to activate that environment before using the tool.**

Create a Virtual Environment on Windows

Guide for virtualenv installation

1. Create the virtual environment:

```
virtualenv -p python3 venv
```

2. Activate the virtual environment:

```
venv\Scripts\activate.bat
```

Create a Virtual Environment on Linux/Mac

Guide for virtualenv installation

1. Create the virtual environment:

```
virtualenv -p python3 .venv
```

2. Activate the virtual environment:

```
source .venv/bin/activate
```

Install M3-CLI

3. To install the `M3-CLI` tool, use the following command:

```
pip install m3-cli
```

Tool Update

Pre-Requisites

The tool update requires the same pre-requisites as installation.

CLI Update Detection Flow

Once a day, when any CLI command is invoked, the CLI performs a check for available updates. This includes making the `m3 health-check` call to the server and using its response to refresh the data stored in the `default.cr` file about the latest CLI version and distribution links.

This data is used between daily update checks to notify users of pending updates while avoiding the need to communicate with the server on each request.

An update notification is printed to the user before the actual response and contains instructions on how to update the CLI client on Linux, Windows, or macOS machines.

If the `m3 health-check` call fails due to a server-side error while the CLI performs a daily update check, a warning is printed informing the user of the error and that the CLI is unable to automatically check for updates.

To force the CLI to check for an update, invoke the `m3 health-check` command manually.

Update Instructions

Windows

1. Check your current version:

```
pip list
```

2. Install the update, using the latest CLI version as m3-cli parameter, for example:

```
pip install m3-cli==3.150.0
```

Example:

```
(venv) C:\Maestro3\m3-cli>pip list
Package            Version
------------------ --------
attrs              23.2.0
...
m3-cli             3.101.10
...
(venv) C:\Maestro3\m3-cli>pip install m3-cli==3.150.0
...
Installing collected packages: m3-cli
  Attempting uninstall: m3-cli
    Found existing installation: m3-cli 3.101.10
    Uninstalling m3-cli-3.101.10:
      Successfully uninstalled m3-cli-3.101.10
Successfully installed m3-cli-3.150.0
```

3. Verify the installation:

```
m3 --help
m3 health-check
```

Linux/Mac

1. Activate your virtual environment (if using one):

```
source .venv/bin/activate
```

2. Check your current version:

```
pip list
```

3. Install the update, using the latest CLI version as m3-cli parameter, for example:

```
pip install m3-cli==3.150.0
```

Example:

```
(.venv) user@localhost:~/maestro3/m3-cli$ pip list
Package            Version
------------------ --------
attrs              23.2.0
...
```

```
m3-cli             3.101.10
...

(.venv) user@localhost:~/maestro3/m3-cli$ pip install m3-cli==3.150.0
Collecting m3-cli==3.150.0
  Downloading m3_cli-3.150.0-py3-none-any.whl (245 kB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 245.8/245.8 kB 2.1 MB/s eta 0:00:00
Installing collected packages: m3-cli
  Attempting uninstall: m3-cli
    Found existing installation: m3-cli 3.101.10
    Uninstalling m3-cli-3.101.10:
      Successfully uninstalled m3-cli-3.101.10
Successfully installed m3-cli-3.150.0
```

4. Verify the installation:

```
m3 --help
m3 health-check
```

Verification Examples

If the `M3-CLI` version is outdated, the `health-check` command will return:

```
(venv) C:\Maestro3\m3-cli>m3 health-check
================================================================================
You are using an outdated version of m3-cli (3.101.10). Version 3.150.0 is now
available.
Please use the next link to download cli for Linux -
https://mcc.cloud.epam.com/cli/3.101.10/m3-cli-x86_64.AppImage
To upgrade cli on macOS run the following command: pip install
https://mcc.cloud.epam.com/cli/3.101.10/m3-cli-3.101.10.zip
For Windows run: pip install https://mcc.cloud.epam.com/cli/3.101.10/m3-cli-
3.101.10.zip
================================================================================

Response:
+-------------------+---------+-------------+------------+---------------------+
| Server Available  | Mails   | Ownership   | Terraform  | Cli Latest Version  |
+===================+=========+=============+============+=====================+
| True              | True    | True        | True       | 3.150.0             |
+-------------------+---------+-------------+------------+---------------------+
```

If the `M3-CLI` version is up to date:

```
$ m3 health-check
Response:
+-------------------+---------+-------------+------------+---------------------+
| Server Available  | Mails   | Ownership   | Terraform  | Cli Latest Version  |
+===================+=========+=============+============+=====================+
| True              | True    | True        | True       | 3.150.0             |
+-------------------+---------+-------------+------------+---------------------+
```

Uninstallation

Pre-Requisites

The tool uninstallation requires the same pre-requisites as installation.

Uninstallation Flow

Windows

1. To uninstall the tool, run the following command:

```
pip uninstall m3-cli
```

Example:

```
(venv) C:\Maestro3_tests\m3-cli-test-updating>pip uninstall m3-cli
Found existing installation: m3-cli 3.150.0
Uninstalling m3-cli-3.150.0:
  Would remove:
    c:\maestro3_tests\m3-cli-test-updating\venv\lib\site-packages\m3_cli-
3.150.0.dist-info\*
    c:\maestro3_tests\m3-cli-test-updating\venv\lib\site-packages\m3cli\*
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\access_meta.json
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\bash_m3cli_complete.sh
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\commands_def.json
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\m3cli_complete.py
    c:\maestro3_tests\m3-cli-test-updating\venv\m3cli\zsh_m3cli_complete.sh
    c:\maestro3_tests\m3-cli-test-updating\venv\scripts\m3.exe
Proceed (Y/n)? y
  Successfully uninstalled m3-cli-3.150.0
```

Linux/Mac

1. Activate your virtual environment (if using one):

```
source .venv/bin/activate
```

2. To uninstall the tool, run the following command:

```
pip uninstall m3-cli
```

Example:

```
(.venv) user@localhost:~/maestro3_tests/m3-cli-test-updating$ pip uninstall m3-cli
Found existing installation: m3-cli 3.150.0
Uninstalling m3-cli-3.150.0:
  Would remove:
    /home/user/maestro3_tests/m3-cli-test-updating/.venv/lib/python3.10/site-
packages/m3_cli-3.150.0.dist-info/*
    /home/user/maestro3_tests/m3-cli-test-updating/.venv/lib/python3.10/site-
packages/m3cli/*
    /home/user/maestro3_tests/m3-cli-test-updating/.venv/m3cli/access_meta.json
    /home/user/maestro3_tests/m3-cli-test-updating/.venv/m3cli/bash_m3cli_complete.sh
    /home/user/maestro3_tests/m3-cli-test-updating/.venv/m3cli/commands_def.json
    /home/user/maestro3_tests/m3-cli-test-updating/.venv/m3cli/m3cli_complete.py
    /home/user/maestro3_tests/m3-cli-test-updating/.venv/m3cli/zsh_m3cli_complete.sh
    /home/user/maestro3_tests/m3-cli-test-updating/.venv/bin/m3
Proceed (Y/n)? y
  Successfully uninstalled m3-cli-3.150.0
```

Tip: Uninstalling `M3-CLI` does not delete all libraries that were installed with `M3-CLI`. For complete removal, delete the `venv` (Windows) or `.venv` (Linux/Mac) directory.

Go to Table of Contents

Usage Guide

Configuration

Before using `M3-CLI`, execute the `m3 access` command to configure all necessary settings.

Automatic Configuration

Run the following command and follow the interactive prompts:

```
m3 access
```

Manual Configuration

To configure settings manually, set up the following environment variables:

- `M3SDK_ACCESS_KEY`: Specifies an M3 access key associated with your Maestro3 user
- `M3SDK_SECRET_KEY`: Specifies the secret key associated with the access key (essentially the “password” for the access key)
- `M3SDK_ADDRESS`: Specifies the address of the Maestro3 environment

Non-Interactive Configuration

Alternatively, you can set credentials non-interactively using the `m3 access` command with specified parameters:

```
(venv) C:\Maestro3\m3-cli>m3 access --access_key <access_key> --secret_key
<secret_key> --api_address <api_address>
```

After running this command, a `default.cr` file with your access parameters will be created in the `.m3cli` folder inside your home directory.

Obtaining Credentials

Obtain Maestro3 URL

Before obtaining your credentials, you need the Maestro3 base URL - the web address of your Maestro3 instance used to configure CLI/SDK connectivity.

How to obtain your Maestro3 URL:

Option 1: From Your Login Page - Use the address you access the Maestro3 web interface with - Format: `https://<your-maestro3-domain>/`

Option 2: Contact Your Administrator - Reach out to your DevOps/Platform team or System Administrator - The deployment team will have this information from the initial setup

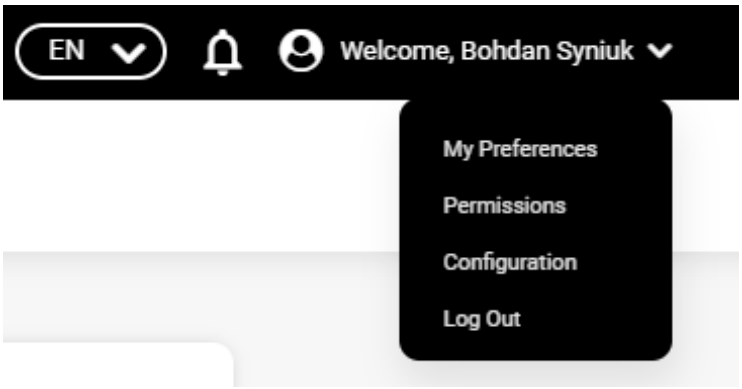
Option 3: Check Internal Documentation - Review your organization’s onboarding materials - Check internal wikis, knowledge bases, or welcome emails

> **Note:** Once you have this URL, you’ll use it as the `--api_address` parameter in the configuration methods described above (Automatic, Manual, or Non-Interactive). Use only the base domain without path suffixes (e.g., remove `/login`).

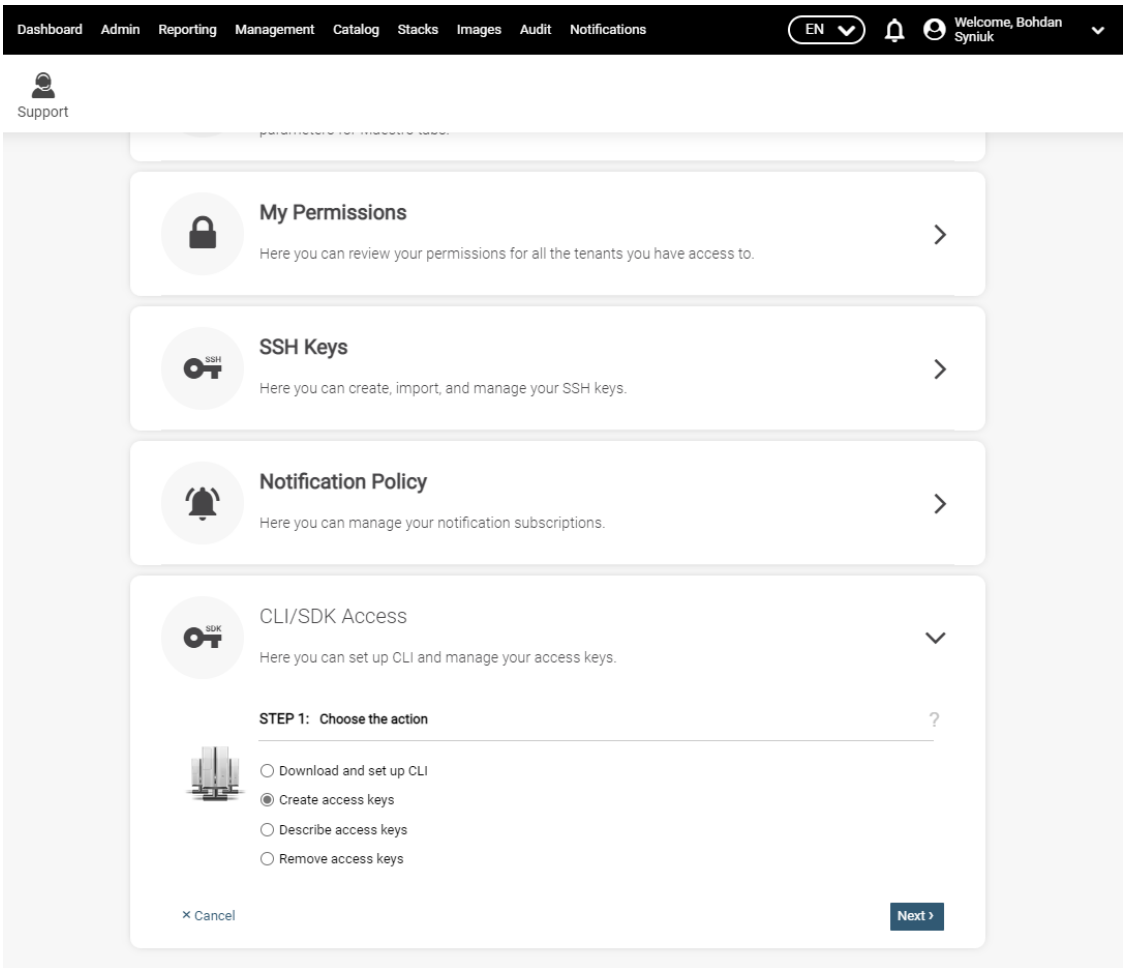
Generating Access Credentials

To obtain your credentials, log in to the Maestro3 UI and follow these steps:

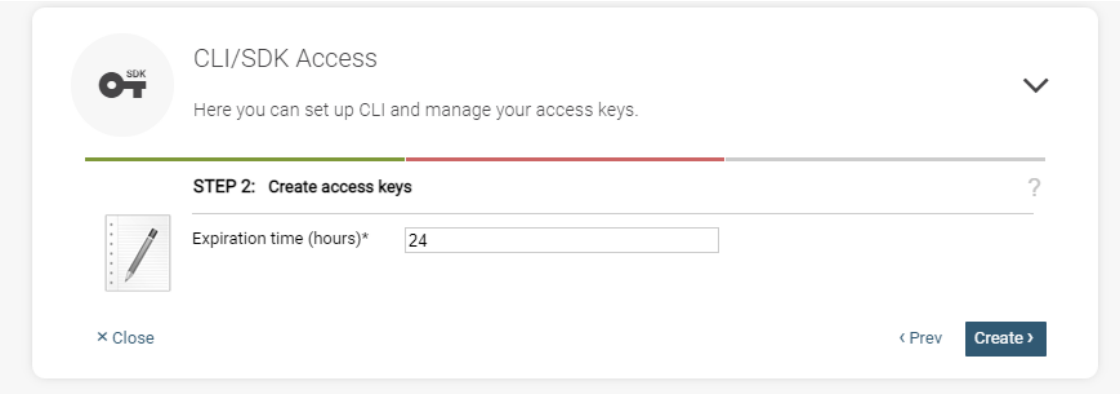
1. Switch to the My Account page:



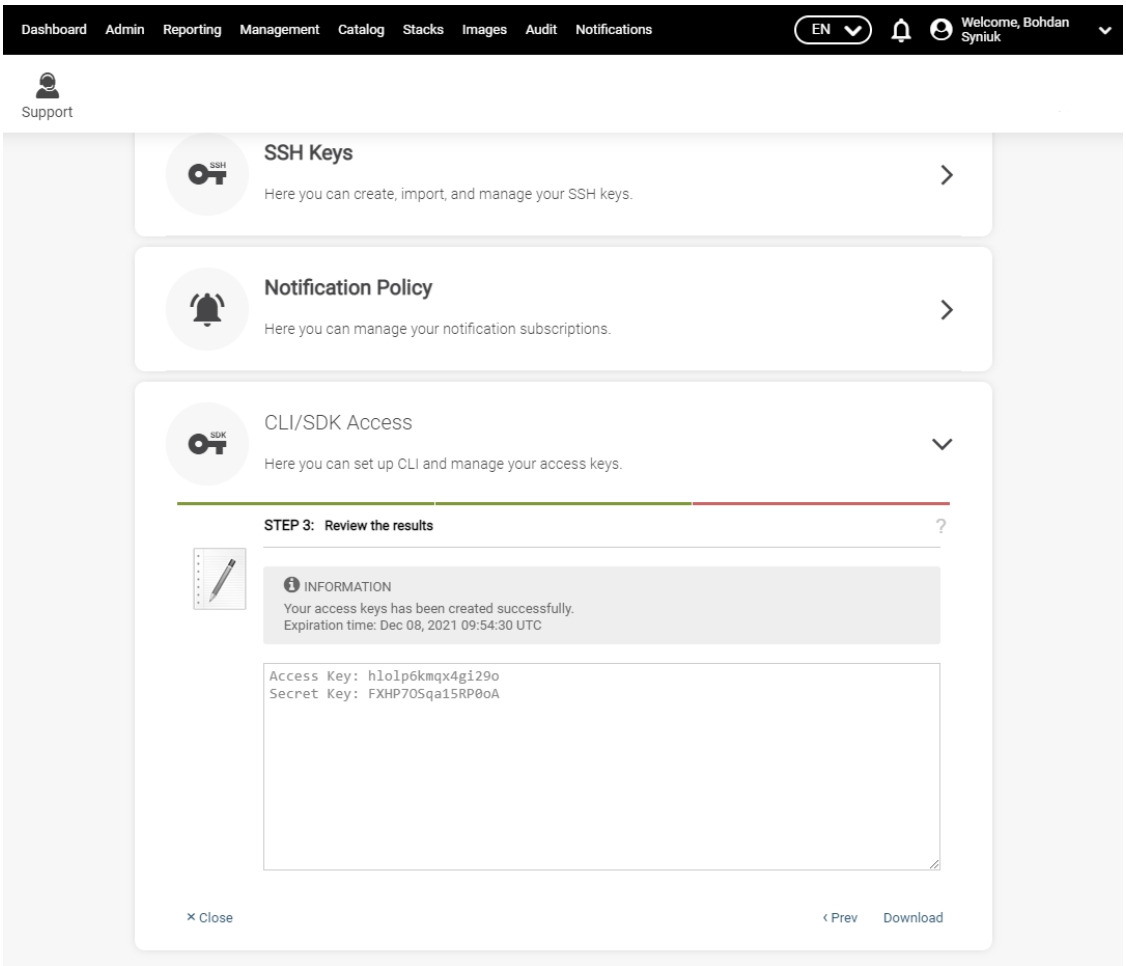
2. Start the 'CLI/SDK Access' wizard:



3. Specify the expiration time for the credentials:



4. Obtain your credentials and export the access key to environment variable `M3SDK_ACCESS_KEY`, and the secret key to `M3SDK_SECRET_KEY`:



Optional Configuration

The following environment variables can be used to override the default values that `M3-CLI` uses:

- `M3CLI_CONFIGURATION_FOLDER_PATH`: Specifies the path to the directory containing files with command and parameter definitions (`commands_def.json`, `commands_help.py`). Default value: internal application path.
- `M3SDK_VERSION`: Specifies the SDK version to use. Default version: `3.150.0`.
- `M3CLI_DEBUG`: Set to `True` to enable debug mode. Default value: `False`.
- `LOG_PATH`: Specifies a custom path to store the `M3-CLI` log file.

Usage Examples

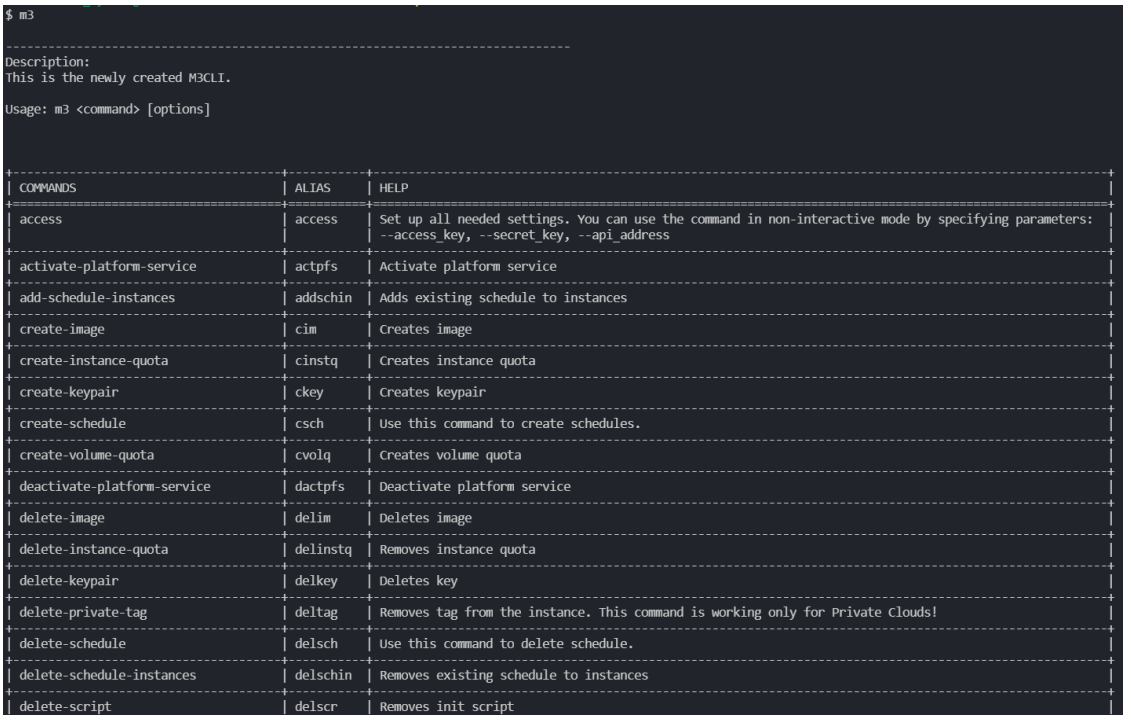
After installing the tool and setting the credentials as environment variables, `M3-CLI` is ready to use.

> **Tip:** The development of Maestro3 CLI is ongoing. Examples below contain demo data and will be updated.

To get information about available commands and parameters, run `m3` as displayed below:

Root Help

The root help displays data about all available commands:



Command Help

The `--help` flag displays abbreviated parameter information:

```
(.venv)
[redacted] MINGW64 /c/data/m3cli/examples/env
$ m3 din --help

Usage: m3 describe-instances(din) [options]

Use --full-help parameter for all available options

Options:

    --region,         -r,      *  The region name
    --tenant-name,    -tn,     *  Tenant name
    --instance-ids,   -iids,      Instances IDs
    --resource-tag,   -tag,       Names of the resource tag: test1key:test1value
```

Full Command Help

Use the `--full-help` flag to get all available options:

```
[redacted] MINGW64 /c/data/m3cli/examples/env
$ m3 din --full-help

--------------------------------------------------------------------------------
Description:

Use this command to return information about instances that you own.

Examples:

1. Describes all available instances for the certain tenant in the specified region

        m3 describe-images -tn <tenantName> -r <region>

2. Describes all instances for the certain tenant in region with specified instance ID

        m3 describe-images -tn <tenantName> -r <region> -ids <instanceId>

3. Describes an instances with specified IDs

        m3 describe-images -tn <tenantName> -r <region> -id <instanceId1,instanceId2>


Usage: m3 describe-instances(din) [options]

Options:

    --help                    Show abbreviated help message and exit.
    --full-help               Show full help message and exit.
    --verbose                 Writes logs into "m3cli.log" file
    --json                    Show the server response in the JSON format
    --table                   Show the server response in the Table format (default)
    --full                    Show the server full response without special formatting (can be used with --json view type)

    --region,         -r,     *  The region name
    --tenant-name,    -tn,    *  Tenant name
    --instance-ids,   -iids,     Instances IDs
    --resource-tag,   -tag,      Names of the resource tag: test1key:test1value
```

Output Modes

Table output mode (default):

```
$ m3 din --region AWS-EUCENTRAL --tenant-name MSTR[redacted] --instance-ids i-0dc9f[redacted]
Response:
+--------------------+--------------+---------+---------+--------------------+---------------+-----------+---------+------------+
| instanceId         | tenant       | state   | shape   | privateIpAddress   | keyName       | imageId   | owner   | platform   |
+====================+==============+=========+=========+====================+===============+===========+=========+============+
| i-0dc9f[redacted]  | AWS-MSTR[redacted] | running | Custom  | 10.0.[redacted]    | [redacted]    | Unknown   |         | Linux      |
+--------------------+--------------+---------+---------+--------------------+---------------+-----------+---------+------------+
```

JSON output mode:

```
$ m3 din --region AWS-EUCENTRAL --tenant-name MSTR[redacted] --instance-ids i-0dc9f368e[redacted] --json
[
    {
        "imageId": "Unknown",
        "instanceId": "i-0dc9f368e[redacted]",
        "keyName": "[redacted]server",
        "platform": "Linux",
        "privateIpAddress": "10.0.[redacted]",
        "publicDnsName": "[redacted]",
        "tenant": "AWS-MSTR[redacted]",
        "state": "running",
        "shape": "Custom"
    }
]
```

Full output mode:

```
$ m3 din --region AWS-EUCENTRAL --tenant-name MSTR[redacted] --instance-ids i-0dc9f3[redacted] --full
Response:


instances:
- architecture = x86_64
  availabilityZone = eu-central-1a
  cloud = AWS
  cpuCount = 1
  creationDate = Mar 24, 2021 17:03:16 UTC
  creationDateTimestamp = 1616605396000
  deviceNames:
  - /dev/sda1
  disableApiTermination = null
  diskInfos:
  - deviceName = /dev/sda1
    diskId = vol-03356918d[redacted]
    system = true
  imageId = Unknown
  instanceId = i-0dc9f368e[redacted]
  instanceName = [redacted]server
  keyName = [redacted]server
  memoryMb = null
  modificationDate = Mar 24, 2021 17:03:16 UTC
  nativeImageId = ami-07670[redacted]
  owner = null
  platform = Linux
  privateDnsName = ip-10-[redacted]eu-central-1.compute.internal
  privateIpAddress = 10.0.[redacted]
  publicDnsName = [redacted]-central-1.compute.amazonaws.com
  publicIpAddress = [redacted]
  region = AWS-EUCENTRAL
  regionId = [redacted]
  securityGroupsNames:
  - [redacted]
  - [redacted]
  - basic
  - [redacted]
  shape = Custom
  state = running
  subnetId = [redacted]
  tags:
  - key = [redacted]
    value = [redacted]server
  tenant = AWS-MSTR[redacted]
  tenantInRegionId = [redacted]
```

Tip: The `--verbose` parameter writes the command result to the terminal.

Go to Table of Contents

Support

How to Report an Issue

When reporting an issue, please provide as much detail as possible, including:

1. **Python version:** The `M3-CLI` tool supports `Python 3.10+`
2. **Clear description:** A concise description of the issue
3. **Steps to reproduce:** Detailed steps to reproduce the issue
4. **Error messages:** Any error messages or logs that can help us understand the problem

Please send your issue report to: `SupportSyndicateTeam@epam.com`

How to Communicate with the Support Team

When communicating with the support team, please be respectful and patient. Our team is here to help you, and we will do our best to resolve your issue as quickly as possible.

Tips for Effective Communication:

1. **Be clear and concise** in your descriptions
2. **Provide all necessary information** related to your issue
3. **Include screenshots or logs** if available

Contact Information

- **Email:** `SupportSyndicateTeam@epam.com`
- **Response Time:** We aim to respond to all queries within 7 calendar days (5 business days, excluding weekends)

Tool Information

- **Company:** EPAM
- **Tool:** M3-CLI
- **Supported Python Versions:** 3.10+
- **Support Contact:** SupportSyndicateTeam@epam.com

Go to Table of Contents

Annexes

Annex 1: Common Use Cases

This section provides practical use cases and workflows for common tasks performed with M3-CLI. Each use case includes step-by-step instructions and examples.

Use Case List

1. Getting Started
2. Instance Lifecycle Management
3. Storage Management
4. SSH Key Management
5. Scheduling and Automation
6. Tagging and Organization
7. Infrastructure as Code with Terraform
8. Cost Management and Reporting
9. Platform Services Management
10. Networking Configuration
11. Monitoring and Insights

Getting Started Use Case

Use Case: First-Time Setup and System Check

Scenario: You've just installed M3-CLI and need to verify your setup and access to the Maestro3 platform.

Steps:

1. Configure your credentials:

```
m3 access --access_key <your_access_key> --secret_key <your_secret_key> --api_address
<api_address>
```

2. Verify system status:

```
m3 health-check
```

Expected output:

```
Response:
+-------------------+---------+-------------+------------+---------------------+
| Server Available  | Mails   | Ownership   | Terraform  | Cli Latest Version  |
+===================+=========+=============+============+=====================+
| True              | True    | True        | True       | 3.150.0             |
+-------------------+---------+-------------+------------+---------------------+
```

3. List available tenants:

```
m3 describe-tenants --cloud AWS
```

4. List available regions for your tenant:

```
m3 describe-regions --tenant <tenant_name>
```

Instance Lifecycle Management

Use Case 1: Launching a New Instance

Scenario: You need to launch a new web server instance in AWS.

Steps:

1. List available images:

```
m3 describe-images --region us-east-1 --tenant <tenant_name> --json
```

2. List available shapes (instance types):

```
m3 describe-shapes --cloud AWS --tenant <tenant_name> --region us-east-1
```

3. Create/Import SSH key if needed:

```
m3 create-key --key-name web-server-key --cloud AWS --region us-east-1 --tenant
<tenant_name>
```

4. Launch the instance:

```
m3 run-instances \
  --image ami-12345678 \
  --name web-server-01 \
  --key-name web-server-key \
  --number-of-instances 1 \
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --shape t3.medium \
  --tag "Environment:Production,Application:WebServer,Owner:TeamA"
```

5. Verify the instance is running:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --running
```

Use Case 2: Managing Instance State

Scenario: You need to stop instances during non-business hours and start them again in the morning.

Steps:

1. Stop an instance:

```
m3 stop-instances --instance-id i-0123456789abcdef0 --tenant <tenant_name> --region
us-east-1
```

2. Verify the instance is stopped:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --instance-id i-
0123456789abcdef0 --stopped
```

3. Start the instance again:

```
m3 start-instances --instance-id i-0123456789abcdef0 --tenant <tenant_name> --region
us-east-1
```

4. Reboot an instance if needed:

```
m3 reboot-instances --instance-id i-0123456789abcdef0 --region us-east-1 --tenant
<tenant_name>
```

Use Case 3: Terminating Instances with Protection

Scenario: You need to terminate a test instance while ensuring production instances are protected.

Steps:

1. Enable termination protection on the production instance:

```
m3 manage-termination-protection \
  --tenant <tenant_name> \
  --region us-east-1 \
  --instance-id i-prod123456 \
  --action enable
```

2. Terminate the test instance:

```
m3 terminate-instances \
  --instance-id i-test123456 \
  --region us-east-1 \
  --tenant <tenant_name>
```

3. Verify termination:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --instance-id i-
test123456
```

Storage Management

Use Case 1: Adding Storage to an Existing Instance

Scenario: Your application instance is running out of disk space and needs additional storage.

Steps:

1. Check current storage:

```
m3 describe-storages --region us-east-1 --tenant <tenant_name> --instance-id i-
0123456789abcdef0
```

2. Create and attach new storage:

```
m3 create-attach-storage \
  --name app-data-vol-02 \
  --instance-id i-0123456789abcdef0 \
  --tenant <tenant_name> \
  --region us-east-1 \
  --size 100
```

3. Verify storage attachment:

```
m3 describe-storages --region us-east-1 --tenant <tenant_name> --instance-id i-
0123456789abcdef0
```

Use Case 2: Moving Storage Between Instances

Scenario: You need to move a data volume from one instance to another.

Steps:

1. Detach storage from the source instance:

```
m3 detach-storage \
  --tenant <tenant_name> \
  --region us-east-1 \
  --instance-id i-source123 \
  --storage-id vol-0123456789
```

2. Wait for detachment to complete, then verify:

```
m3 describe-storages --region us-east-1 --tenant <tenant_name> --storage-id vol-
0123456789
```

3. Attach the volume to the target instance:

> **Note:** Use the Maestro3 UI or appropriate cloud-specific commands to attach the detached volume to the new instance.

Use Case 3: Storage Cleanup

Scenario: Remove unused storage volumes to reduce costs.

Steps:

1. List all storage volumes:

```
m3 describe-storages --region us-east-1 --tenant <tenant_name> --json
```

2. Identify unattached volumes and delete them:

```
m3 delete-storage \
  --tenant <tenant_name> \
  --region us-east-1 \
  --storage-id vol-unused123
```

SSH Key Management

Use Case: Setting Up SSH Access for a Team

Scenario: You need to set up SSH keys for team members across multiple regions.

Steps:

1. Import an existing team member's public key:

```
m3 import-key \
  --file-path /path/to/john_doe_key.pub \
  --key-name john-doe-key \
  --cloud AWS \
  --region us-east-1 \
  --tenant <tenant_name>
```

2. Create a new key for automation purposes:

```
m3 create-key \
  --key-name automation-key \
  --cloud AWS \
  --region us-east-1 \
  --tenant <tenant_name>
```

3. List all available keys:

```
m3 describe-keys --cloud AWS --region us-east-1 --tenant <tenant_name>
```

4. Replicate the key to another region:

```
m3 manage-key \
  --cloud AWS \
  --key-name automation-key \
  --region us-west-2 \
  --action add \
  --tenant <tenant_name>
```

5. Remove unused keys:

```
m3 delete-key --key-name old-unused-key
```

Scheduling and Automation

Use Case 1: Creating Cost-Saving Schedules

Scenario: You want to automatically stop development instances at 7 PM and start them at 8 AM on weekdays.

Steps:

1. Create a stop schedule for the evening:

```
m3 create-schedule \
  --action STOP \
  --cloud AWS \
  --region us-east-1 \
  --cron "0 0 19 * * MON-FRI" \
  --description "Stop dev instances at 7 PM weekdays" \
  --name dev-evening-stop \
  --schedule-type "My instances with tag" \
  --tenant <tenant_name> \
  --tag-key Environment \
  --tag-value Development
```

2. Create a start schedule for the morning:

```
m3 create-schedule \
  --action START \
  --cloud AWS \
  --region us-east-1 \
  --cron "0 0 8 * * MON-FRI" \
  --description "Start dev instances at 8 AM weekdays" \
  --name dev-morning-start \
  --schedule-type "My instances with tag" \
  --tenant <tenant_name> \
  --tag-key Environment \
  --tag-value Development
```

3. Verify the schedules:

```
m3 describe-schedules --cloud AWS --tenant <tenant_name> --region us-east-1
```

Use Case 2: Managing Instance-Specific Schedules

Scenario: Add or remove schedules from specific instances.

Steps:

1. Add a schedule to specific instances:

```
m3 add-schedule-instances \
  --cloud AWS \
  --region us-east-1 \
  --tenant <tenant_name> \
  --instance-id i-0123456,i-0123457,i-0123458 \
  -dn dev-evening-stop
```

2. Remove a schedule from instances:

```
m3 delete-schedule-instances \
  --cloud AWS \
  --region us-east-1 \
  --instance-id i-0123456 \
  -dn dev-evening-stop \
  --tenant <tenant_name>
```

3. Delete a schedule entirely:

```
m3 delete-schedule --region us-east-1 --name dev-evening-stop --tenant <tenant_name>
```

Tagging and Organization

Use Case: Implementing a Tagging Strategy

Scenario: You need to tag instances according to your organization's tagging policy for cost allocation and management.

Steps:

1. Tag a new instance during launch:

```
m3 run-instances \
  --image ami-12345678 \
  --name app-server-01 \
  --key-name my-key \
  --number-of-instances 1 \
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --shape t3.medium \
  --tag "Project:CustomerPortal,Environment:Production,Owner:john.doe@company.com"
```

2. Add tags to existing instances:

```
m3 set-tags \
  --cloud AWS \
  --instance-id i-0123456789abcdef0 \
  --tenant <tenant_name> \
  --region us-east-1 \
  --tag "Compliance:PCI-DSS,BackupPolicy:Daily"
```

3. View all tags on an instance:

```
m3 describe-tags \
  --cloud AWS \
  --instance-id i-0123456789abcdef0 \
  --tenant <tenant_name> \
  --region us-east-1
```

4. Find all instances with a specific tag:

```
m3 describe-instances \
  --region us-east-1 \
  --tenant <tenant_name> \
  --resource-tag "Environment:Production"
```

5. Find untagged instances:

```
m3 describe-instances \
  --region us-east-1 \
```

```
  --tenant <tenant_name> \
  --group UNTAGGED
```

6. Remove specific tags:

```
m3 delete-tags \
  --cloud AWS \
  --instance-id i-0123456789abcdef0 \
  --tenant <tenant_name> \
  --region us-east-1 \
  --tag "OldTag,AnotherOldTag"
```

Infrastructure as Code with Terraform

Use Case 1: Deploying Infrastructure from Terraform Template

Scenario: You need to deploy a complete application stack using a Terraform template.

Steps:

1. Upload the Terraform template:

```
m3 upload-terraform-template \
  --source-path /path/to/terraform/template \
  --terraform-version v1.5.0 \
  --cloud AWS \
  --tenant <tenant_name> \
  --template web-app-stack \
  --variables "environment:production,instance_count:3,instance_type:t3.large"
```

2. Generate a variable file for review:

```
m3 generate-platform-service-varfile \
  --cloud AWS \
  --region us-east-1 \
  --tenant <tenant_name> \
  --service web-app-stack
```

3. Plan the deployment (dry-run):

```
m3 plan-terraform-template \
  --cloud AWS \
  --template web-app-stack \
  --tenant <tenant_name> \
  --variables "environment:production,instance_count:3,instance_type:t3.large"
```

4. Apply the template:

```
m3 apply-terraform-template \
  --cloud AWS \
  --template web-app-stack \
  --tenant <tenant_name>
```

5. Check deployment status:

```
m3 describe-terraform-stacks \
  --cloud AWS \
  --tenant <tenant_name>
```

Use Case 2: Managing Terraform Template Locks

Scenario: Prevent concurrent modifications to critical infrastructure.

Steps:

1. Lock the template before making changes:

```
m3 lock-terraform-template \
  --cloud AWS \
  --template production-infrastructure \
  --tenant <tenant_name> \
  --expiration 2
```

2. Make your changes and prolong the lock if needed:

```
m3 prolong-terraform-template-lock \
  --cloud AWS \
  --template production-infrastructure \
  --tenant <tenant_name> \
  --expiration 1
```

3. Unlock when done:

```
m3 unlock-terraform-template \
  --cloud AWS \
  --template production-infrastructure \
  --tenant <tenant_name>
```

Use Case 3: Template Cleanup

Scenario: Remove unused Terraform templates and their resources.

Steps:

1. List all templates:

```
m3 describe-terraform-templates --cloud AWS --tenant <tenant_name>
```

2. Destroy resources created by the template:

```
m3 destroy-terraform-stack \
  --cloud AWS \
  --template old-test-stack \
  --tenant <tenant_name>
```

3. Delete the template:

```
m3 delete-terraform-template \
  --cloud AWS \
  --template old-test-stack \
  --tenant <tenant_name>
```

Cost Management and Reporting

Use Case 1: Monthly Cost Analysis

Scenario: Generate comprehensive cost reports for the current month.

Steps:

1. Get total costs for the month:

```
m3 total-report \
  --tenant-group <tenant_group_name> \
  --year 2024 \
  --month 03
```

2. Get a detailed breakdown by service:

```
m3 subtotal-report \
  --tenant-group <tenant_group_name> \
  --year 2024 \
  --month 03
```

3. Get resource-level details:

```
m3 resource-report \
  --tenant-group <tenant_group_name> \
  --year 2024 \
  --month 03 \
  --day 15
```

4. Export to JSON for further analysis:

```
m3 resource-report \
  --tenant-group <tenant_group_name> \
  --year 2024 \
  --month 03 \
  --json > monthly_costs.json
```

Use Case 2: Identifying Cost Optimization Opportunities

Scenario: Find resources that are driving costs and can be optimized.

Steps:

1. Generate a cost and usage report:

```
m3 cost-usage-report \
  --cloud AWS \
  --date 15.03.2024 \
  --region us-east-1 \
  --resource-type INSTANCE \
  --tenant <tenant_name>
```

2. Find untagged resources (potential waste):

```
m3 untagged-resource-report \
  --tenant <tenant_name> \
  --from 01.03.2024 \
  --to 31.03.2024
```

3. Review insights for specific instances:

```
m3 describe-insights \
  --instance-id i-0123456789abcdef0 \
  --cloud AWS \
  --region us-east-1 \
  --tenant <tenant_name>
```

4. Check budget status:

```
m3 budgets-report \
  --tenant-group <tenant_group_name> \
  --year 2024 \
  --month 03
```

Use Case 3: Multi-Tenant Cost Reporting

Scenario: Generate cost reports across multiple tenants for management review.

Steps:

1. Generate a multi-tenant report:

```
m3 multitenant-report \
  --from 01.03.2024 \
  --to 31.03.2024 \
  --region us-east-1 \
  --report-type summary \
  --include-billing-source \
  --json > multitenant_costs.json
```

Platform Services Management

Use Case: Publishing and Deploying a Platform Service

Scenario: You've developed a custom service and want to make it available to your tenants.

Steps:

1. Upload the service script:

```
m3 upload-script \
  --cloud AWS \
  --source-path /path/to/init-script.sh \
  --script monitoring-agent-installer \
  --tenant <tenant_name>
```

2. Publish the service:

```
m3 publish-platform-service \
  --service monitoring-service \
  --service-version 2.1.0 \
  --template monitoring-template \
  --discoverable-url https://docs.company.com/monitoring \
  --title "Enterprise Monitoring Service" \
  --supported-cloud AWS,AZURE,GOOGLE \
  --cloud AWS \
  --tenant <tenant_name> \
  --provider-version v1.5.0 \
  --category Monitoring,Security \
  --description "Comprehensive monitoring solution" \
  --icon-path /path/to/icon.png
```

3. Add usage documentation:

```
m3 add-service-section \
  --service monitoring-service \
  --section USAGE \
  --block-title "Getting Started" \
  --block-value "Install the agent on your instances using the provided script..."
```

4. Activate the service for a tenant:

```
m3 activate-platform-service \
  --cloud AWS \
  --region us-east-1 \
  --tenant <tenant_name> \
  --service monitoring-service
```

5. Verify service activation:

```
m3 describe-platform-service-stacks \
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --service monitoring-service
```

6. Deactivate when needed:

```
m3 deactivate-platform-service \
  --cloud AWS \
  --region us-east-1 \
  --tenant <tenant_name> \
  --service-id <service_id>
```

Networking Configuration

Use Case 1: Setting Up Static IPs

Scenario: Configure static IP addresses for your instances.

Steps:

1. Allocate a static IP:

```
m3 allocate-ip \
  --tenant <tenant_name> \
  --region us-east-1 \
  --network vpc-0123456789
```

2. List available IPs:

```
m3 describe-ip --tenant <tenant_name> --region us-east-1
```

3. Associate the IP with an instance:

```
m3 associate-ip \
  --tenant <tenant_name> \
  --region us-east-1 \
  --static-ip 203.0.113.25 \
  --instance-id i-0123456789abcdef0
```

4. Disassociate when needed:

```
m3 disassociate-ip \
  --tenant <tenant_name> \
  --region us-east-1 \
  --static-ip 203.0.113.25
```

5. Release unused IP:

```
m3 release-ip \
  --tenant <tenant_name> \
  --region us-east-1 \
  --ip 203.0.113.25
```

Use Case 2: VLAN Management

Scenario: Set up network segmentation using VLANs.

Steps:

1. List available VLANs:

```
m3 describe-vlans --tenant <tenant_name> --region us-east-1
```

2. Activate a VLAN:

```
m3 activate-vlan \
  --tenant <tenant_name> \
  --region us-east-1 \
  --vlan-name production-vlan-100 \
  --description "Production environment network"
```

3. Move an instance to the VLAN:

```
m3 move-to-vlan \
  --tenant <tenant_name> \
  --region us-east-1 \
  --vlan-name production-vlan-100 \
  --instance-id i-0123456789abcdef0
```

4. Deactivate VLAN when no longer needed:

```
m3 deactivate-vlan \
  --tenant <tenant_name> \
  --region us-east-1 \
  --vlan-name old-dev-vlan-50
```

Monitoring and Insights

Use Case 1: Monitoring Instance Health and Performance

Scenario: Regular monitoring and insights gathering for your infrastructure.

Steps:

1. List all running instances:

```
m3 describe-instances \
  --region us-east-1 \
  --tenant <tenant_name> \
  --running
```

2. Get detailed insights for a specific instance:

```
m3 describe-insights \
  --instance-id i-0123456789abcdef0 \
  --cloud AWS \
  --region us-east-1 \
  --tenant <tenant_name>
```

3. Check instances with high-risk factors:

```
m3 describe-instances \
  --region us-east-1 \
  --tenant <tenant_name> \
  --insights \
  --group VULNERABLE
```

4. View recent events:

```
m3 describe-events \
  --tenant <tenant_name> \
  --cloud AWS \
  --number-of-events 50 \
  --search-type RESOURCE \
  --resource-id i-0123456789abcdef0 \
  --region us-east-1
```

Use Case 2: Resource Inventory and Compliance

Scenario: Maintain an accurate inventory of all resources for compliance purposes.

Steps:

1. Get a complete resource inventory:

```
m3 describe-resources \
  --region us-east-1 \
  --tenant <tenant_name> \
  --json > resource_inventory.json
```

2. Filter by specific tags:

```
m3 describe-resources \
  --region us-east-1 \
  --tenant <tenant_name> \
  --tag "Compliance:SOC2"
```

3. Find untagged resources:

```
m3 describe-resources \
  --region us-east-1 \
  --tenant <tenant_name> \
  --group UNTAGGED
```

4. Check instances by owner:

```
m3 describe-instances \
  --region us-east-1 \
  --tenant <tenant_name> \
  --owner john.doe@company.com
```

Use Case 3: Creating Custom Images

Scenario: Create reusable images from configured instances.

Steps:

1. Create an image from an instance:

```
m3 create-image \
  --name web-server-baseline-v2 \
  --instance-id i-0123456789abcdef0 \
  --region us-east-1 \
```

```
  --tenant <tenant_name> \
  --description "Web server with Apache 2.4, PHP 8.1, and security hardening"
```

2. List available images:

```
m3 describe-images --region us-east-1 --tenant <tenant_name>
```

3. Delete an old image:

```
m3 delete-image \
  --image-id ami-old123456 \
  --region us-east-1 \
  --tenant <tenant_name>
```

Advanced Workflows

Workflow 1: Complete Application Deployment

Scenario: Deploy a complete 3-tier web application with proper tagging, monitoring, and cost controls.

Steps:

1. Set up SSH keys:

```
m3 create-key --key-name app-deployment-key --cloud AWS --region us-east-1 --tenant
<tenant_name>
```

2. Deploy the database tier:

```
m3 run-instances \
  --image ami-database \
  --name app-db-01 \
  --key-name app-deployment-key \
  --number-of-instances 1 \
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --shape r5.xlarge \
  --tag "Tier:Database,Application:CustomerApp,Environment:Production"
```

3. Deploy the application tier:

```
m3 run-instances \
  --image ami-app-server \
  --name app-server \
  --key-name app-deployment-key \
  --number-of-instances 3 \
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --shape c5.large \
  --tag "Tier:Application,Application:CustomerApp,Environment:Production"
```

4. Deploy the web tier:

```
m3 run-instances \
  --image ami-web-server \
  --name web-server \
  --key-name app-deployment-key \
  --number-of-instances 2 \
```

```
  --cloud AWS \
  --tenant <tenant_name> \
  --region us-east-1 \
  --shape t3.medium \
  --tag "Tier:Web,Application:CustomerApp,Environment:Production"
```

5. Set up cost-saving schedules for non-production environments (if deploying dev/test):

```
m3 create-schedule \
  --action STOP \
  --cloud AWS \
  --region us-east-1 \
  --cron "0 0 19 * * MON-FRI" \
  --description "Stop non-prod instances at 7 PM" \
  --name non-prod-evening-stop \
  --schedule-type "My instances with tag" \
  --tenant <tenant_name> \
  --tag-key Environment \
  --tag-value Development
```

6. Monitor the deployment:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --resource-tag
"Application:CustomerApp" --running
```

Workflow 2: Disaster Recovery Exercise

Scenario: Create backups and prepare for disaster recovery.

Steps:

1. Create images of critical instances:

```
m3 create-image \
  --name prod-db-backup-2024-03-15 \
  --instance-id i-prod-db-01 \
  --region us-east-1 \
  --tenant <tenant_name> \
  --description "Production database backup before upgrade"
```

2. Document the current state:

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --json > pre-dr-
state.json
m3 describe-storages --region us-east-1 --tenant <tenant_name> --json > pre-dr-
storage.json
```

3. Set up a backup service:

```
m3 backup \
  --tenant <tenant_name> \
  --region us-east-1 \
  --cloud AWS \
  --service-id backup-svc-01 \
  --instance-id i-prod-db-01 \
  --backup-server-id backup-srv-01
```

4. Verify backups and generate reports:

```
m3 describe-events \
  --tenant <tenant_name> \
```

```
--cloud AWS \
--number-of-events 100 \
--search-type BACKUP \
--region us-east-1
```

Best Practices

- **Always use JSON output for automation:**

```
m3 describe-instances --region us-east-1 --tenant <tenant_name> --json | jq
'.instances[] | select(.state=="running")'
```

- **Use tags consistently:**
  - Environment (Production, Development, Testing)
  - CostCenter
  - Owner
  - Application
  - Compliance requirements
- **Implement schedules for cost savings:**
  - Stop non-production instances outside business hours
  - Use appropriate instance sizes
- **Perform regular monitoring:**

```
# Add to cron or scheduled task
m3 describe-instances --region us-east-1 --tenant <tenant_name> --insights --
json > daily-insights-$(date +%Y%m%d).json
```

- **Use variable files for Terraform templates** to maintain consistency across environments.
- **Enable termination protection** on production instances.
- **Conduct regular cost reviews:**

```
# Monthly cost review script
m3 total-report --tenant-group <tenant_group> --year $(date +%Y) --month $(date
+%m) --json > monthly-costs.json
```

Go to Table of Contents

Annex 2: Developers Guide

Who This Guide Is For

This guide is intended for:

- **Platform Engineers** who need to extend or customize M3-CLI functionality
- **DevOps Teams** who want to integrate M3-CLI into their automation workflows
- **System Administrators** who need to understand the internal workings of M3-CLI
- **Contributors** who wish to add new commands or modify existing ones
- **Integration Developers** who are building tools that interact with M3-CLI

When You Need This Guide

You should refer to this guide if you:

1. **Need to add new commands** to M3-CLI for your organization
2. **Want to customize command behavior** through integration plugins
3. **Are developing automation scripts** that leverage M3-CLI's architecture
4. **Need to understand** how M3-CLI processes commands and communicates with the Maestro3 platform
5. **Want to contribute** to the M3-CLI project
6. **Are troubleshooting** command behavior or creating custom output formatters

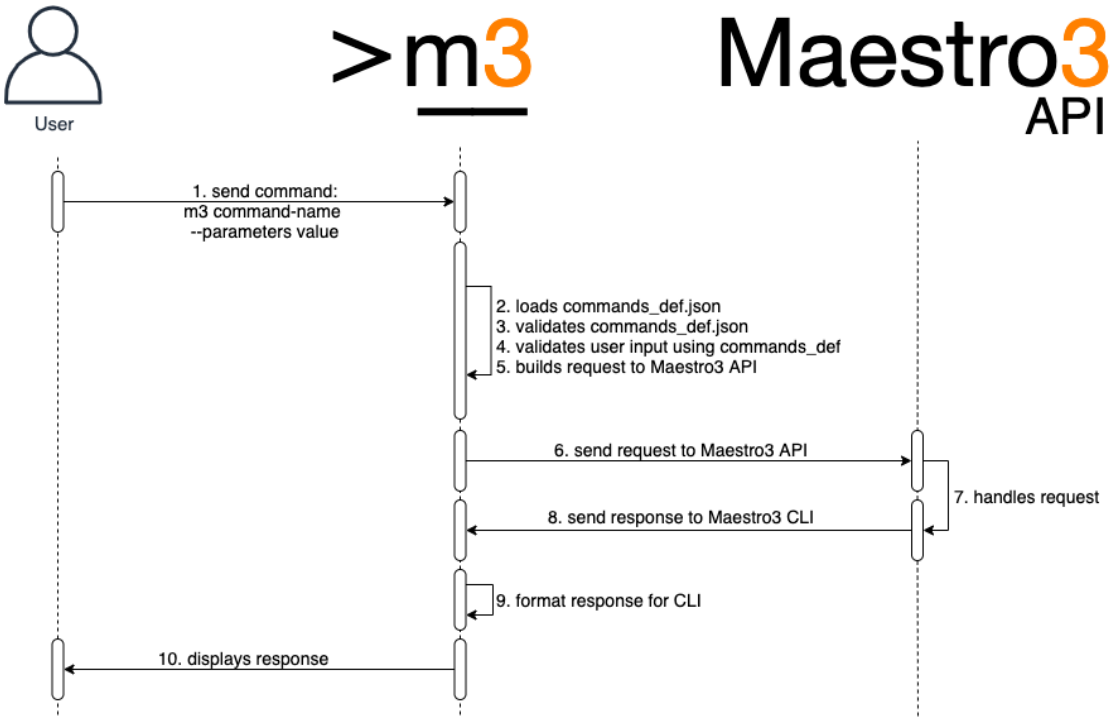
> **Note:** If you're only using M3-CLI to manage your infrastructure, you can skip this section and refer to the Usage Guide and Annex 1: Common Use Cases sections instead.

Architecture Overview

The `M3-CLI` tool is designed to provide a dynamic command-line interface based on command configurations declared in the `commands_def.json` file.

The tool DOES NOT perform any business logic related to management of resources that Maestro3 provides.

The sequence diagram is displayed below:



Commands Definition File

The commands definition file (`commands_def.json`) defines a set of commands, their groupings, and parameters that can be executed via CLI. Here is a full example of attributes found in the definition file:

```
{
    "groups": [
        "A name of a group of related commands"
    ],
    "domain_parameters": {
        "domain-parameter-name-1": {
            "alias": "param_alias",
            "api_param_name": "The name of the parameter that the server accepts to
form the response",
            "help": "Parameter description",
            "required": true,
            "validation": {
                "type": "string"
            }
        }
    },
    "commands": {
        "command-name-1": {
            "api-action": "REQUIRED. Mapping for M3API command",
            "help_file": true,                      // A Flag. If commands help
                                                    // stored into the file
            "help": "Explanation of what command does",
            "alias": "Alias for the command name. Could be used instead of command-
name-1",
            "integration_request": true,            // A Flag. Specify if the
                                                    // request will be processed

            "integration_response": true,           // A Flag. Specify if the
                                                    // response will be processed

            "integration_suffix": "",               // if specified, m3cli should
                                                    // build the plugin name
                                                    // according to the
                                                    // ${command_name}_
                                                    // ${integration_suffix}.py
                                                    // pattern
            "groups": [
                "group1",                           // A name of the group of
                                                    // commands to which this
                                                    // command relates

                "cli-<command-name>-help",          // Defines a unidirectional
                                                    // relation to the <command-
                                                    // name> command

                "email-<notification_type>-group",  // Defines a relation to the
                                                    // Maestro notifications of
                                                    // the type <notification_type>
            ],
            "parameters": {                         // Optional
                "inherited-param-from-domain": {
```

```
                    "parent": "Name of the domain parameter to inherit properties."
                },
                "param-name-1": {
                    "parent": "name",               // The name of the parent
                    "alias": "Alias for the parameter name. Could be used instead of
param-name-1",
                    "api_param_name": "The name of the parameter that the server
accepts to form the response",
                    "help": "REQUIRED. Explanation of what the parameter means",
                    "required": true,               // Set the parameter as a
                                                    // REQUIRED

                    "secure": true,                 // Hide the value of the
                                                    // parameter in logs. Allowed
                                                    // values: [true, false]

                    "validation": {                 // REQUIRED. Set of
                                                    // validation rules.

                        "type": "string/object",    // REQUIRED. The type of
                                                    // parameter. Allowed values:
                                                    // ['string', 'number',
                                                    // 'list', 'object', 'date',
                                                    // 'bool', 'file']

                        "allowed_values": [],       // A list of allowed values.
                                                    // Applicable to param of
                                                    // types 'string' and 'list'

                        "regex": "Regular Expression. Applicable to params of types:
'string', 'file'",
                        "regex_error": "A message to show if the regex check failed.
Applicable to params of types: 'string', 'file'",
                        "properties": {},           // jsonschema validation
                                                    // rules. Applicable to param
                                                    // of type: 'object';

                        "min_value": 0,             // number validation rule.
                                                    // Applicable only to type:
                                                    // 'number',

                        "max_value": 7,             // number validation rule.
                                                    // Applicable only to type:
                                                    // 'number',

                        "max_size_bytes": 1024,     // file validation rule.
                                                    // Applicable only to type:
                                                    // 'file',

                        "file_extensions": ['.txt'] // file validation rule.
                                                    // Applicable only to type:
                                                    // 'file'

                    },
                    "case": "upper/lower"           // convert a value to upper
```

```
                                                    // case automatically
                }
            },
            "output_configuration": {               // REQUIRED. Contains
                                                    // configuration for the
                                                    // output.

                "response_table_headers": [         // Required. Contains list
                  "header-1", "header-2"            // of attributes to display
                ],                                  // in output.
                "none": true,                       // Replace the response from
                                                    // server to "The command has
                                                    // been executed successfully"

                "nullable": true,                   // Set the flag 'nullable' in
                                                    // order to prevent hiding
                                                    // zeros and False values of
                                                    // number and boolean
                                                    // attributes.

                "multiple_table": true,             // A flag. If the response
                                                    // consists of several tables

                // The structure of the "response_table_headers" if the
                // "multiple_table" was specified
                "response_table_headers": [
                  {
                    // The display name of a table
                    "display_name": "Instance price model:",
                    // The name of the table that the server sends in the response
                    "name": "instancePriceModel",
                    // Contains list of attributes to display in output.
                    "headers": []
                  }
                ],
                // Optional. Used to alter appearance of data in columns.
                "headers_customization": {
                  // The header name
                  "the_name_of_header": {
                    // Custom name of the header
                    "header_display_name": "Custom name of the header",
                    // A flag. Disables automatic conversion of string-values to
                    // numbers.
                    "disable_numparse": true,
                    // A flag. Disables automatic alignment of list values to a
                    // column.
                    "prevent_list_formatting": true
                  }
                },
              "unmap_key": "Contains 'key' to extract response if needed."
            }
        }
    },
    "version": "major.minor"     // see the versioning rules below
}
```

Tip: Command help values can be stored in a file named `commands_help.py`. The format for this storage type is Python native. Example:

```
run-instance = """
Use this command to run an instance.

Examples:
1. Describes all available instances for a certain tenant in the specified region
    m3 run-instance -tn <tenant-name> -r <region> -iname <instance-name> -shname
<shape-name> -key <key-name> -count <number-of-instances>
"""
```

Related Commands Section in "full-help" Contents

The related commands section is constructed automatically based on the `groups` attribute of each command. Related commands are displayed as:

```
Related commands:

1. Deletes a schedule from your tenant's schedule library:
        m3 delete-schedule -r <region> -tn <tenant> -n <name>
```

There are two ways a command can be added to the list of related commands of another command.

Approach 1: Bidirectional Group Relations

Declare a group of commands in the `group` attribute in the root of the `commands_def` file, and then assign it to all commands within that group. All commands within the group share a bidirectional relation with each other.

Example:

```
{
  "groups": ["group_A"],
  "commands": {
    "first_command": {
      "groups": ["group_A"],
      // ...
    },
    "second_command": {
      "groups": ["group_A"],
      // ...
    }
  }
}
```

With this configuration, `first_command` will be in the list of related commands of `second_command` and vice versa.

Approach 2: Unidirectional Auxiliary Commands

Define a unidirectional relation between commands by adding the name of the other command to the list of groups with a special prefix and suffix (`cli-` and `-help`).

Example:

```
{
  "commands": {
    "first_command": {
      "groups": ["cli-second-command-help"],
      // ...
    },
    "second_command": {
```

```
      // ...
    }
  }
}
```

With this configuration, `first_command` will appear in the list of related commands of `second_command`. However, `second_command` will not be in the list of related commands of `first_command`.

Default Command Parameter Values

Default values for command parameters can be stored in the `m3.properties` file:

```
...
tenant-name = SFTL-SLCTL
region = SFTL-OPENSTACK-SLCTL
...
```

You can create several `m3.properties` files in different directories with different default values. To select the required parameters, change the current working directory to the directory containing the appropriate `m3.properties` file.

Alternatively, store default values for command parameters in the `default.cr` file:

```
{
    ...
    "tenant-name": "SFTL-SLCTL",
    "region": "SFTL-OPENSTACK-SLCTL",
    ...
}
```

Default parameter values from the `m3.properties` file take precedence over values from the `default.cr` file.

Integration Request/Response

Since version `2.1.0`, it is possible to process CLI input and server output manually using custom Python code. To use this feature, you need to create a plugin with the appropriate properties:

0. Create a directory named `plugins` at the path specified as the value for the environment variable `M3CLI_CONFIGURATION_FOLDER_PATH`.

1. Create a simple Python module (`filename.py`) and place it in the `plugins` directory mentioned above.

2. To modify CLI input, create a method inside your script named `create_custom_request` that receives one parameter, `request`:

```
def create_custom_request(request):
    return request
```

3. To modify CLI output, create a method inside your script named `create_custom_response` that receives `request` and `response` parameters:

```
def create_custom_response(request, response):
    return response
```

> **Tip:** The type of the return value in the functions described above should be the same as the received parameter (`request`/`response`)

4. After successful plugin creation, add the appropriate attributes to the command description in the file `commands_def.json`: `"integration_request": true` or `"integration_response": true`

Nullable Boolean Fields

Since version `3.41.7`, it is possible to display boolean fields even when their value is `False`. To use this feature:

Add the `nullable` field with the value `true` to the `output_configuration` section of the `commands_def.json` file:

```
"output_configuration": {
        "nullable": true,
        "response_table_headers": [
          "region",
          "nativeName",
          "cloud",
          "active",
          "hidden"
        ]
      }
```

In the example above, if the value of the `active` field is `false`, the value in the table column will not be hidden.

Command Output Formatting

When the CLI receives a response with items that have empty attributes, these attributes will be dropped from the response (in table view only).

To add custom formatting to a response attribute, add the `headers_customization` parameter to `output_configuration`:

```
"output_configuration": {
    "response_table_headers": ["header-1", "header-2"],
    "headers_customization": {
      "header-1": {
        "disable_numparse": true,
        "prevent_list_formatting": true
      }
    }
}
```

- The `disable_numparse` setting disables automatic conversion of string values to numbers.
- The `prevent_list_formatting` setting disables automatic alignment of list values to a column.

Interactive Options

`Interactive options` is a feature that enables a command to fetch additional parameters from the server and either prompt a user to interactively provide values for these parameters or create a file with additional parameters (a `varfile`) for future use.

Fetching Additional Parameters

To enable a command to fetch additional parameters, add the following attribute to the command's definition:

```
"interactive_options": {
    // The name of parameters group
    "option_name": "params",
    // The name of server handler that returns the list of additional parameters
    "parameters_handler": "GET_SERVICE_VARIABLES_INFO",
    // The name of server handler that validates the list of additional parameters
    "validation_handler": "VALIDATE_SERVICE_VARIABLES",
}
```

If this feature is enabled, the command will: 1. Get additional parameters via the `parameters_handler` 2. Ask the user to fill in values for these parameters or provide a varfile 3. Validate the additional parameters 4. Send them back to the server as part of the original request

Generating Variable Files

To enable a command to create a varfile, add the following attribute to the command's definition:

```
"interactive_options": {
    // The name of server handler that returns the list of additional parameters
    "parameters_handler": "GET_SERVICE_VARIABLES_INFO",
    // Marks the command as a generator of a file with additional parameters
    // fetched with the "parameters_handler"
    "generate_varfile": true,
}
```

The `varfile` is a JSON file that contains all the additional parameters that the `parameter_handler` provides. The parameters are either filled with default values or have empty values.

M3-CLI Autocomplete

By default, `M3-CLI` includes autocomplete functionality that supports only Unix-based platforms (bash & zsh interpreters).

Activating Autocomplete

1. Create a virtual environment.

2. Install `M3-CLI`.

3. Create a symlink to the virtual environment:

```
sudo ln -s your_path_to_venv/bin/m3 /usr/local/bin/m3
```

4. Start a new terminal session.

5. Execute the command:

```
sudo m3 enable-autocomplete
```

6. Restart the terminal session.

Deactivating Autocomplete

1. Execute the command:

```
sudo m3 disable-autocomplete
```

2. Restart the terminal session.

Go to Table of Contents